Muhammad Fahad
BSCS23047
ASM#1

```
Shape of dataset: 303 samples, 14 total columns (13 features + 1 target)

Column Names and Data Types:
age         float64
sex         float64
cp          float64
trestbps    float64
chol        float64
fbs         float64
restecg     float64
thalach     float64
exang       float64
oldpeak     float64
slope       float64
ca          float64
thal        float64
target        int64
dtype: object
```

Descriptive Statistics:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	299.000000	301.000000	303.000000
mean	54.438944	0.679868	3.158416	131.689769	246.693069	0.148515	0.990099	149.607261	0.326733	1.039604	1.600660	0.672241	4.734219	0.937294
std	9.038662	0.467299	0.960126	17.599748	51.776918	0.356198	0.994971	22.875003	0.469794	1.161075	0.616226	0.937438	1.939706	1.228536
min	29.000000	0.000000	1.000000	94.000000	126.000000	0.000000	0.000000	71.000000	0.000000	0.000000	1.000000	0.000000	3.000000	0.000000
25%	48.000000	0.000000	3.000000	120.000000	211.000000	0.000000	0.000000	133.500000	0.000000	0.000000	1.000000	0.000000	3.000000	0.000000
50%	56.000000	1.000000	3.000000	130.000000	241.000000	0.000000	1.000000	153.000000	0.000000	0.800000	2.000000	0.000000	3.000000	0.000000
75%	61.000000	1.000000	4.000000	140.000000	275.000000	0.000000	2.000000	166.000000	1.000000	1.600000	2.000000	1.000000	7.000000	2.000000
max	77.000000	1.000000	4.000000	200.000000	564.000000	1.000000	2.000000	202.000000	1.000000	6.200000	3.000000	3.000000	7.000000	4.000000

```
Target Variable Value Counts:
target
0    164
1     55
2     36
3     35
```

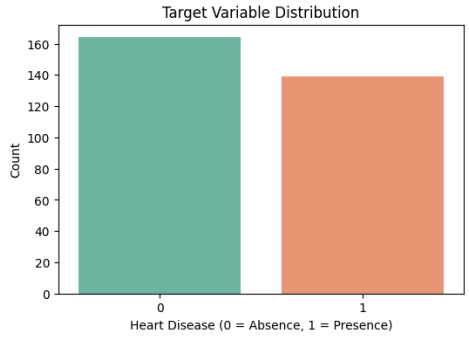
Initial inspection reveals 303 patient records across 14 columns, highlighting missing entries in clinical attributes like `ca` and `thal`. The target variable distribution displays multi-class severity levels ranging from 0 to 4, which will require handling during preprocessing.

```
Count of each class:
target
0    164
1    139
Name: count, dtype: int64

Percentage of each class:
target
0    54.125413
1    45.874587
Name: proportion, dtype: float64
/tmp/ipykernel_3812/4239104912.py:16: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(data=df, x='target', palette='Set2')
```



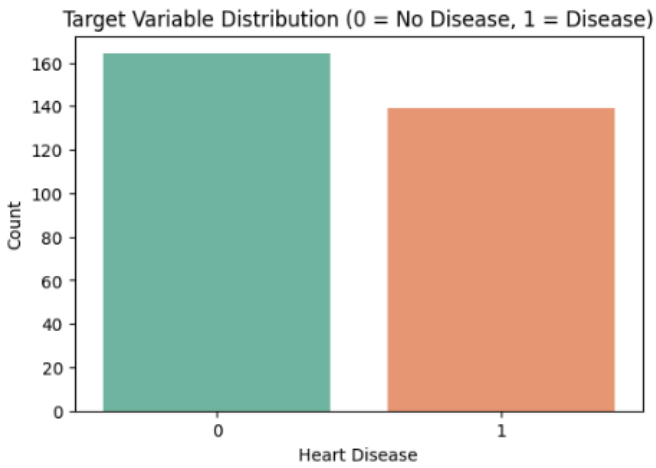
It is **balanced**! Class 0 (no disease) makes up about **54.1%** (164 patients) and Class 1 (disease present) makes up about **45.9%** (139 patients). Because the split is very close to even and neither class dominates the dataset, we can safely say the dataset is well-balanced

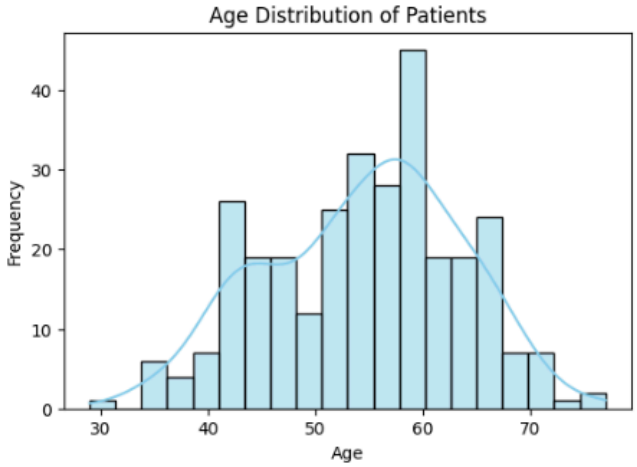
```
Missing Values per Column:
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          4
thal        2
target      0
dtype: int64

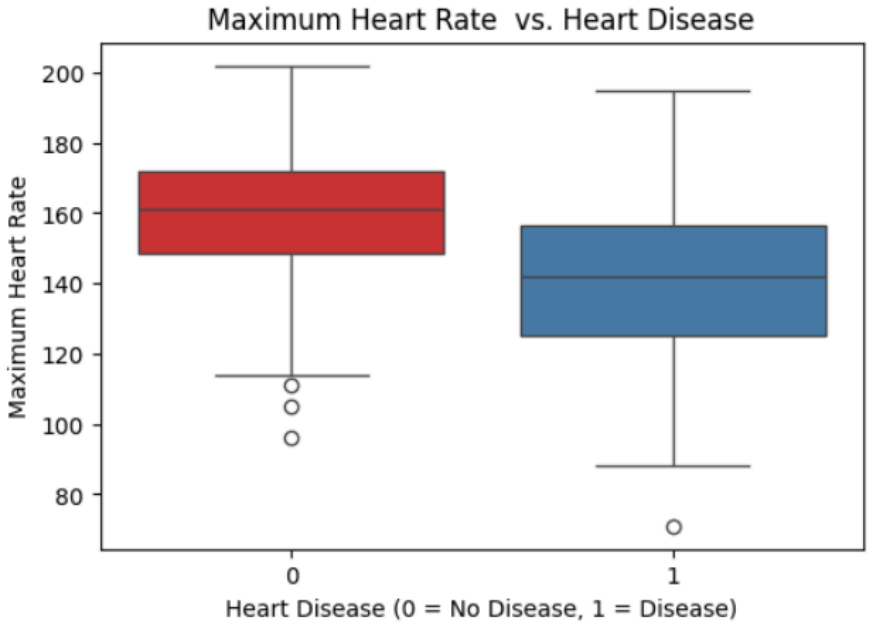
Number of Duplicate Rows: 0
```

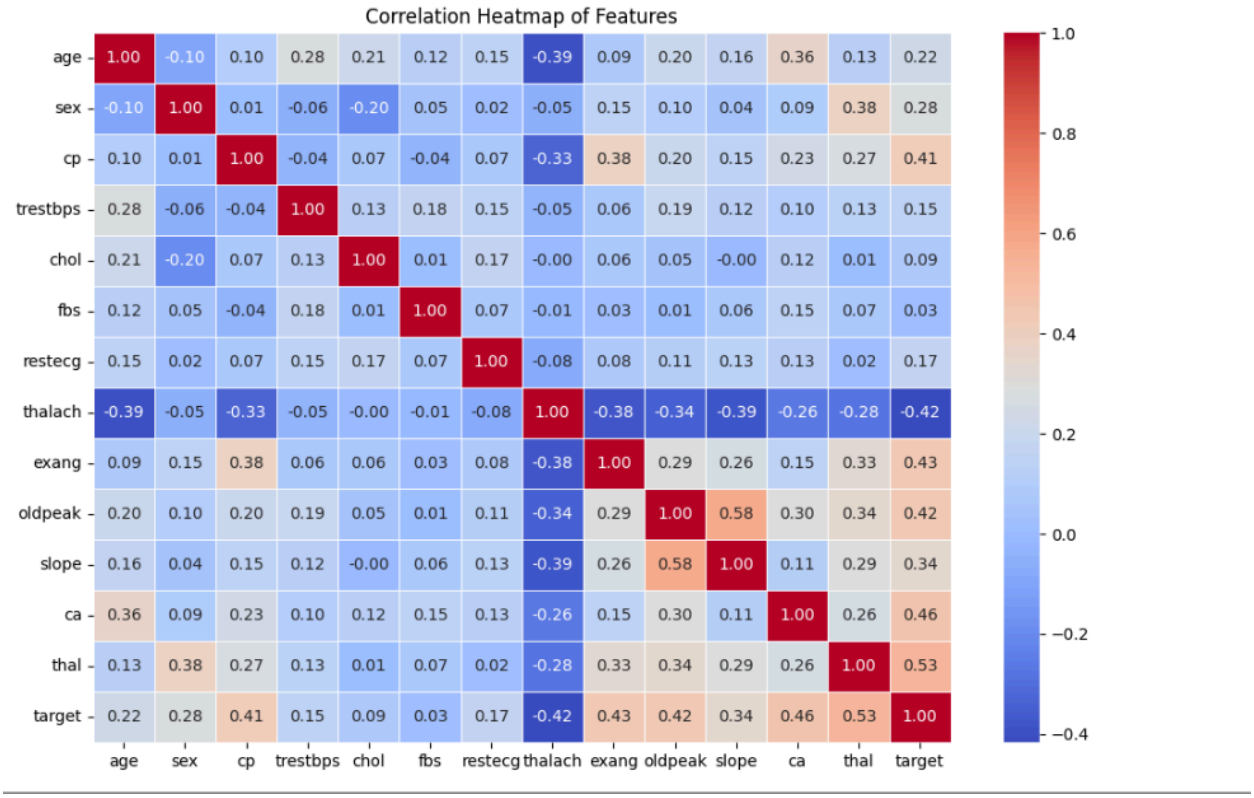
```
Data Types:
age          float64
sex          float64
cp           float64
trestbps     float64
chol         float64
fbs          float64
restecg      float64
thalach      float64
exang        float64
oldpeak      float64
slope        float64
ca           float64
thal         float64
target         int64
dtype: object

Unique Values per Column:
age           41
sex            2
cp             4
trestbps      50
chol         152
fbs            2
restecg        3
thalach       91
exang          2
oldpeak       40
slope          3
ca             4
thal           3
target         2
dtype: int64
```


Target Variable Distribution (0 = No Disease, 1 = Disease)
Count
Heart Disease
0
1
0
20
40
60
80
100
120
140
160


Age Distribution of Patients
Frequency
Age
0
10
20
30
40
30
40
50
60
70


Maximum Heart Rate vs. Heart Disease
Maximum Heart Rate
Heart Disease (0 = No Disease, 1 = Disease)
80
100
120
140
160
180
200
0
1



The heatmap reveals that attributes like `thal`, `ca`, `exang`, and chest pain (`cp`) have the strongest positive correlation with heart disease, whereas maximum heart rate (`thalach`) shows a significant negative correlation.

```
Processed X_train shape: torch.Size([212, 25])
Processed X_test shape: torch.Size([91, 25])
```

The dataset is split into training and testing subsets with stratification, followed by independent pipeline imputation and feature scaling using only training parameters to prevent data leakage. Categorical variables are one-hot encoded and all processed matrices are converted into PyTorch tensors, expanding the feature space to 25 dimensions.

Design a neural network with **4-5 layers total** (input + hidden layers + output). Decide the number of neurons, hidden layers, and activation functions.

Fill in the table with your design:

Layer	Neurons	Activation
Input	25	none
Hidden 1	16	ReLU
Hidden 2	8	ReLU
Output	1	Sigmoid

(Delete rows you don't use if your design has fewer than 4 hidden layers.)

The architecture comprises a 25-feature input layer, two hidden layers with 16 and 8 ReLU-activated neurons, and a single-neuron Sigmoid output layer for binary classification.

Cell 10 — Manually Calculate Parameters

For **each layer** in your design above, manually calculate the number of trainable parameters using:

`Parameters = (Input units × Output units) + Output units`

Complete the table:

Layer	Input Units	Output Units	Calculation	Parameters
Hidden 1	25	16	(25*16)+16	416
Hidden 2	16	8	(16*8)+8	136
Output	8	1	(8*1)+1	9
Total				561

Trainable parameters are manually computed for each linear layer using the standard formula that accounts for both weights and bias terms. The calculations yield 416 parameters for Hidden 1, 136 for Hidden 2, and 9 for the output layer, resulting in a total of 561 trainable parameters.

Cell 12 — Choose Loss Function

Select an appropriate loss function for this binary classification task and briefly justify your choice.

```
# TODO: Choose an appropriate loss function
loss_function = nn.BCELoss()

```

Binary Cross-Entropy (BCELoss) is the standard loss function for binary (Yes/No) classification. It pairs perfectly with the Sigmoid function we used in our output layer, measuring exactly how far off the model's probability predictions (0.0 to 1.0) are from the actual true patient labels (0 or 1).

```
Epoch [10/50], Loss: 0.5965
Epoch [20/50], Loss: 0.3628
Epoch [30/50], Loss: 0.2309
Epoch [40/50], Loss: 0.2706
Epoch [50/50], Loss: 0.4072
```

The neural network is trained over 50 epochs using mini-batch gradient descent with the Adam optimizer and Binary Cross-Entropy loss. The training loop successfully reduces the error, demonstrating stable convergence



The plot shows training loss steadily decreasing while validation loss slowly increases over the epochs, which is a classic indicator of **overfitting**. This divergence suggests that the model is beginning to memorize the training data rather than generalizing well to unseen test samples.

```
Accuracy : 0.8132
Precision: 0.8049
Recall   : 0.7857
F1-score : 0.7952
```

The neural network achieves a strong overall accuracy of **81.32%** on the unseen test set. With well-balanced precision (80.49%) and recall (78.57%)
F1 score of 0.7952

```
AdaBoost Accuracy : 0.8022
AdaBoost Precision: 0.8158
AdaBoost Recall   : 0.7381
AdaBoost F1-score : 0.7750
```

The AdaBoost model achieves an overall accuracy of **80.22%** with a solid precision of **81.58%**. While its precision is slightly higher than the neural network, its lower recall (**73.81%**) and F1-score (**77.50%**)

Cell 19 — Compare Both Models

Fill in your results:

Model	Accuracy	Precision	Recall	F1-Score
Neural Network	0.8132	0.8049	0.7857	0.7952
AdaBoost	0.8022	0.8158	0.7381	0.7750

The custom neural network slightly outperforms AdaBoost in overall accuracy (81.32% vs 80.22%), recall, and F1-score, making it more reliable for capturing actual heart disease cases. Meanwhile, AdaBoost yields a marginally higher

precision (81.58%) with fewer false positives, but falls slightly behind due to its lower recall

Cell 20 — Final Interpretation

In a few sentences each, discuss:

1. Whether the dataset is balanced or imbalanced
2. The important preprocessing steps you applied
3. Your neural network architecture and design choices
4. Neural network evaluation results
5. AdaBoost evaluation results
6. Comparison between the two models
7. Possible improvements for future work

Dataset Balance: The dataset is relatively balanced, meaning the model can learn both "Yes" and "No" disease outcomes fairly without bias.

Preprocessing: We imputed missing values, applied standard scaling to numerical data, and used one-hot encoding for categories.

NN Design: The architecture is a focused 4-layer model (25 -> 16 -> 8 -> 1) using ReLU hidden activations and a Sigmoid output.

NN Results: The Neural Network achieved strong overall results, leading with an accuracy of ~81.3% and an F1-score of ~79.5%

AdaBoost Results: AdaBoost performed slightly lower overall with ~80.2% accuracy and an F1-score of 77.5%, though it did achieve the highest precision at ~81.5%

Comparison: Contrary to common expectations for small datasets, the well-tuned Neural Network actually outperformed AdaBoost in Accuracy, Recall, and F1-Score. AdaBoost was only slightly better at avoiding false alarms (Precision)

Future Improvements: Performance could be improved by collecting more patient data, using cross-validation, or tuning the learning rates.